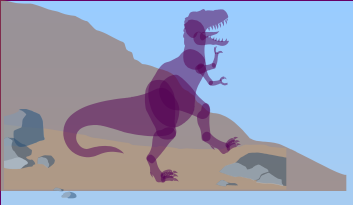
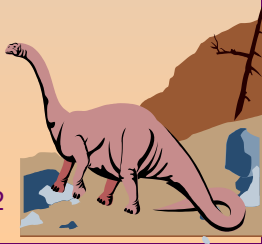
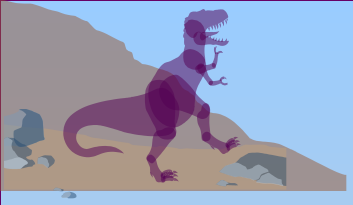


System calls

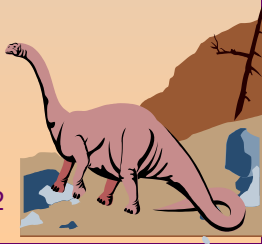
- System calls provide the interface between a running program and the operating system.
  - Generally available as assembly language instructions.
  - Languages defined to replace assembly language for systems programming allow system calls to be made directly (e.g., C, C++)



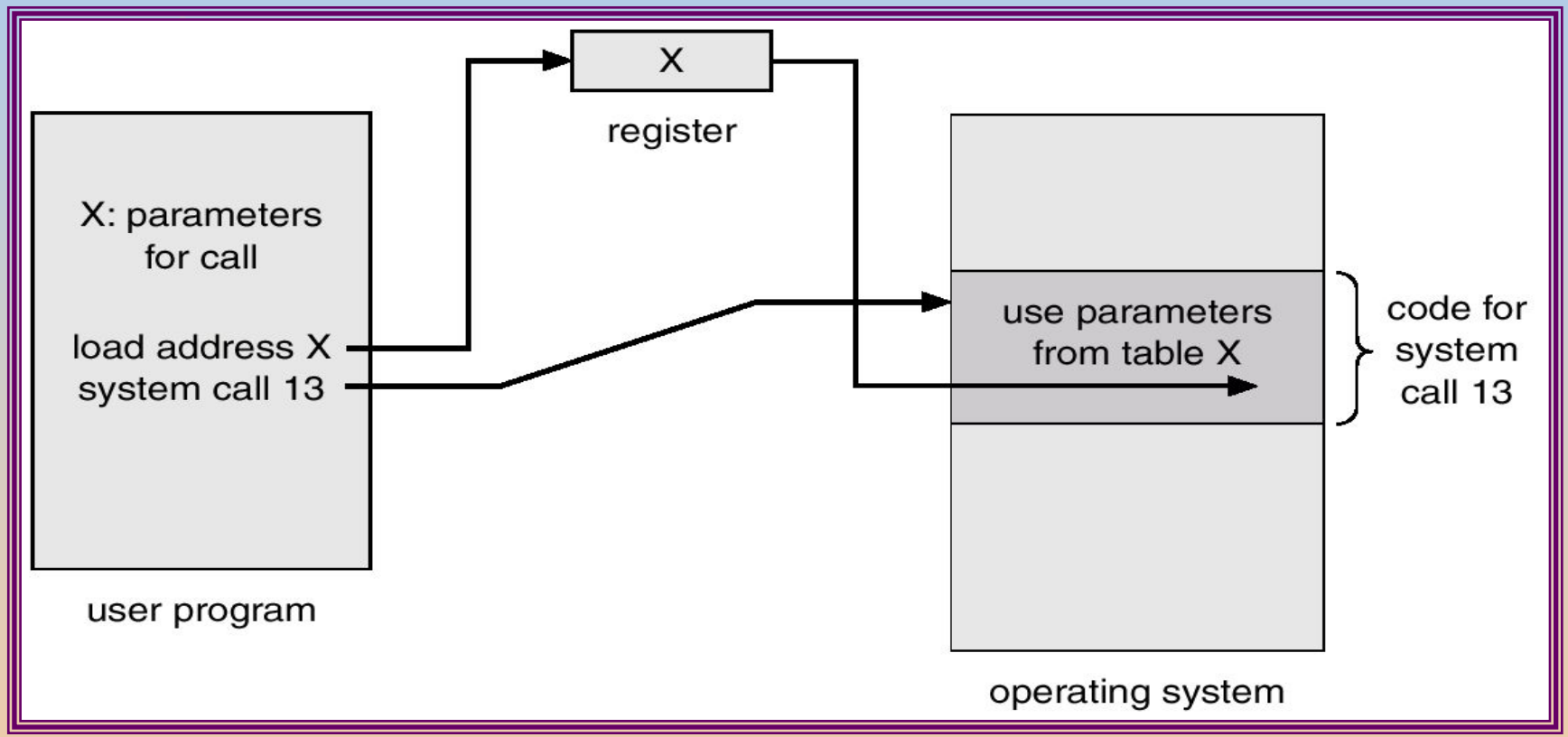


System calls

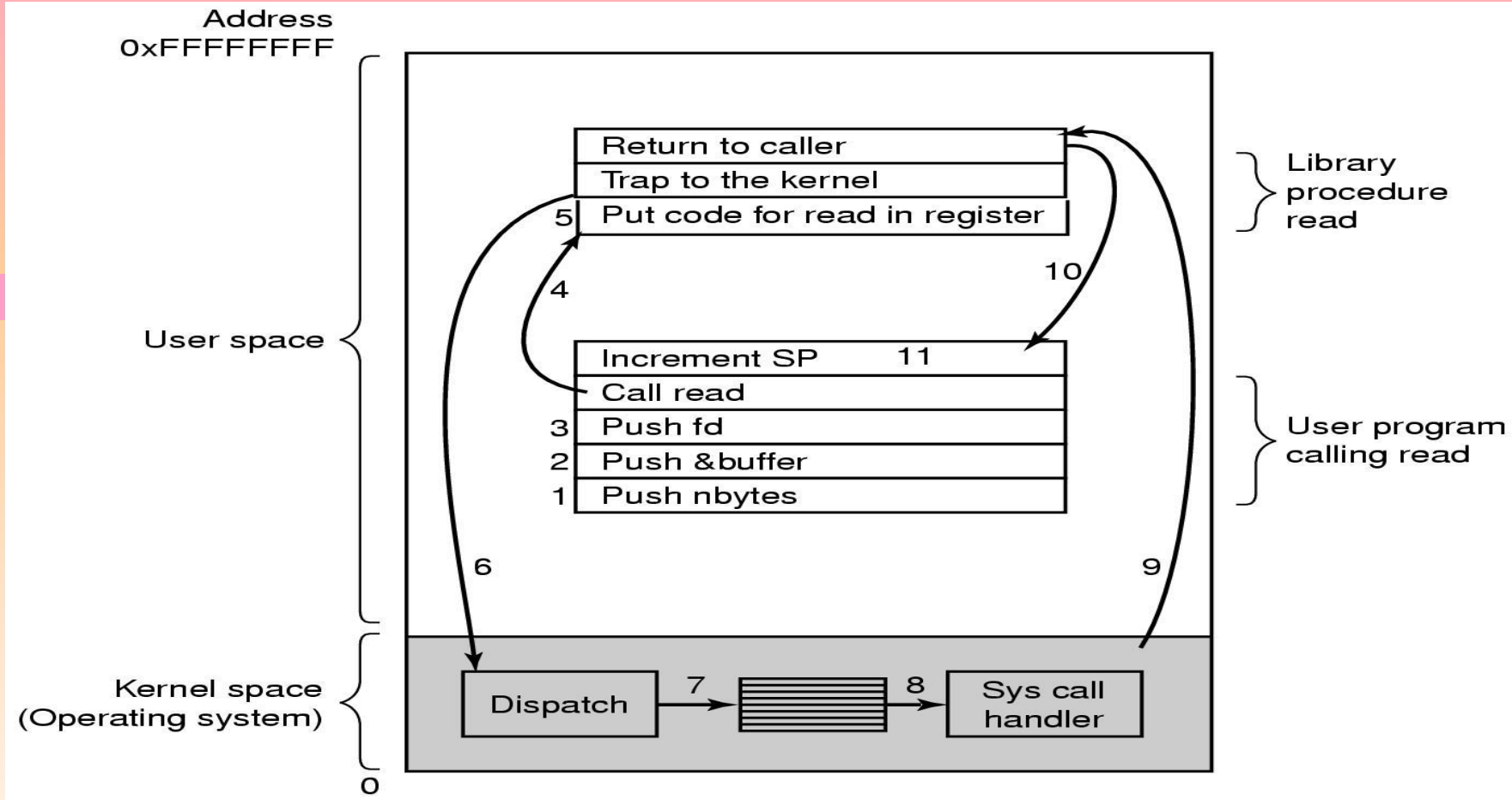
- Three general methods are used to pass parameters between a running program and the operating system.
  - Pass parameters in *registers*.
  - Store the parameters in a table in memory, and the table address is passed as a parameter in a register.
  - *Push* (store) the parameters onto the *stack* by the program, and *pop* off the stack by operating system.



Passing of Parameters As A Table



HOW A SYSTEM CALL WORKS



There are 11 (or more) steps in making the system call

read (fd, buffer, nbytes)

User Space

1. **Push nbytes :** The user program pushes the number of bytes to be read onto the stack.

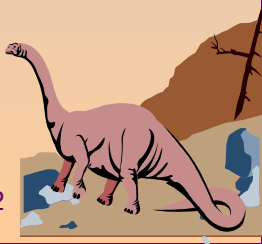
2. **Push &buffer :** The address of the buffer where data will be stored is pushed onto the stack.

3. **Push fd and Call read :** The file descriptor (fd) is pushed, and the program calls the library function read().

4. **Enter Library Procedure :** Control transfers to the library routine for read().

5. **Put System Call Code in Register :** The library procedure places the system call number (for read) into a CPU register.

6. Trap to Kernel :A **trap instruction** is executed. CPU switches from **User Mode** to **Kernel Mode**. Control passes to the operating system.



Kernel Space

7. Dispatch

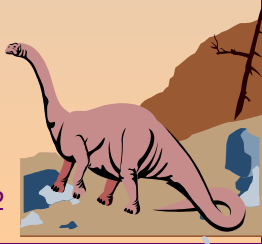
The kernel examines the system call number and uses it to locate the correct service routine.

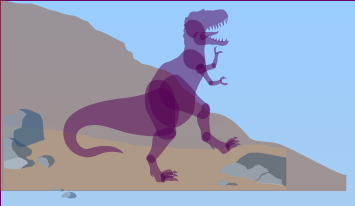
8. System Call Handler

The system call handler executes the requested operation (reads data from the file/device into the buffer).

9. Return from Kernel

After completing the read operation, the kernel returns control to the library procedure in user space.





Back to User Space

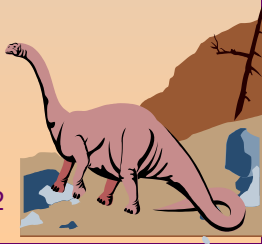
10. Return from Library Procedure

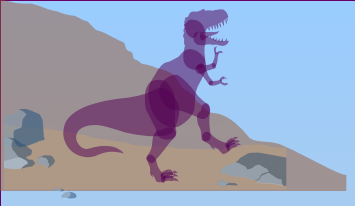
The library routine returns the result (number of bytes read or an error code) to the user program.

11. Increment Stack Pointer (SP)

The stack pointer is adjusted to remove the arguments that were pushed earlier.

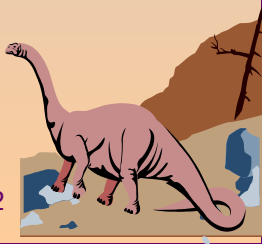
The user program continues execution.

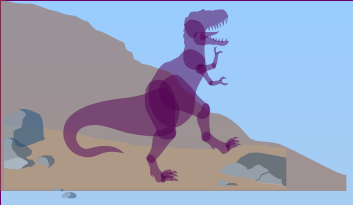




Types of System Calls

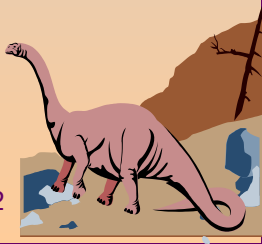
- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection

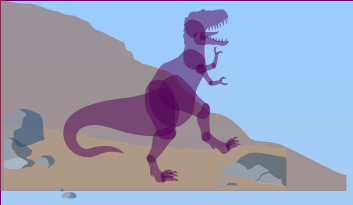




Types of System Calls

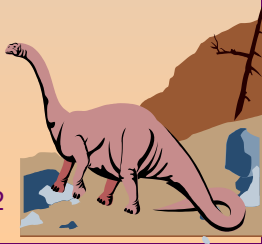
- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes

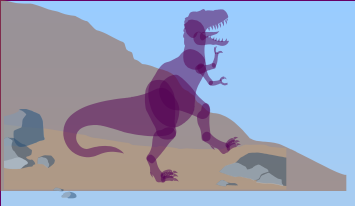




Types of System Calls (Cont.)

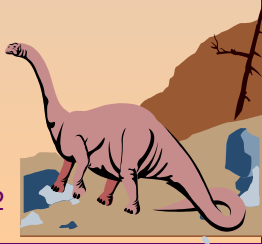
- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

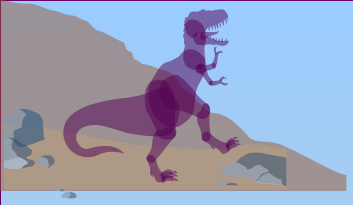




Types of System Calls (Cont.)

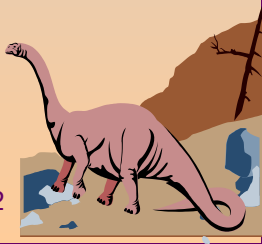
- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**





Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access



Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()